

꽃판: GO!

화투 48장으로 짓고땡을 지어 점수를 내는 싱글 플레이 로그라이크 덱빌더.

AI 활용 기술 문서

A I U S A G E & T E C H N I C A L R E P O R T

기획 · 규칙 설계 · 구현 · 아트 · 테스트 전 공정에 AI를 사용했다

런타임에는 AI를 호출하지 않는다 — 시드로 완전히 재현되는 순수 함수 엔진

AI 산출물은 124개의 자동 테스트를 통과해야만 저장소에 남는다

사용한 프롬프트 원문과 후처리 절차를 저장소에 그대로 기록했다

주요 도구

OpenAI Codex · Claude

이미지

OpenAI 이미지 생성

런타임 AI

사용 안 함

요약 — AI를 어디에 썼고, 어디에 쓰지 않았나

이 프로젝트에서 AI는 **제작 도구**이지 **실행 부품**이 아니다. 기획을 코드로 옮기고, 규칙의 정합성을 검사하고, 카드 그림을 그리고, 경계값 테스트를 만드는 데 AI를 썼다. 반대로 게임이 실행되는 동안에는 어떤 AI API도 호출하지 않는다.

단계	AI가 한 일	사람이 결정한 것
기획 검증	규칙 초안의 모순·사각지대 지적, 대안 제시	짓고땡 채택, 고·스톱 위험 구조, 2~5장 자유 제출로 일반화
규칙 엔진	족보 판정·점수 파이프라인·고 문턱을 순수 함수로 구현	점수식 월 합 $\times$ 배수, 판정 우선순위, 초과 달성 방지 장치
콘텐츠	부적·비결서·화공패·두목 카탈로그 작성과 수치 배분	콘텐츠 종류와 개수, 보상의 성격(배수·자원·규칙)
UI	컴포넌트 구현, 한 화면 레이아웃, 스포트라이트 튜토리얼	드래그 없음, 족보 선택 UI 없음, 스크롤 없음
아트	화투 48장 아틀라스와 카드 자산 생성	도트 화풍, 원본 화투의 구조·모티프 보존, 최종 채택 여부
테스트	경계값·상태 전이·완전 탐색 밸런스 프로브 작성	무엇을 불변식으로 잠글지
런타임	없음. 생성형 AI·유료 API를 실행 중에 호출하지 않는다	

런타임 AI를 배제한 이유

① 재현성

모든 런은 문자열 시드와 RNG 커서로 완전히 재현된다. 같은 시드를 넣으면 카드 순서도, 장터 진열도 똑같이 나온다. 런타임에 모델 호출이 섞이면 이 성질이 즉시 깨지고, 버그 재현도 밸런스 측정도 불가능해진다.

② 심사 환경

API 키·계정·과금이 필요 없어야 심사자가 링크만 열어서 바로 플레이할 수 있다. 최초 로딩 이후에는 네트워크가 끊겨도 게임이 멈추지 않는다.

③ 검증 가능성

규칙 엔진이 결정론적 순수 함수라 124개의 테스트로 전부 잠글 수 있다. AI가 만든 코드라도 **테스트를 통과하지 못하면 저장소에 남지 않는다.**

요약하면 — AI는 이 프로젝트의 생산성을 만들었고, 결정론적 엔진과 자동 검증이 그 결과물의 신뢰성을 만들었다. 콘텐츠 157종과 규칙 엔진을 해커톤 기간 안에 완성할 수 있었던 것은 둘을 함께 썼기 때문이다.

AI 활용 구조

"사람이 결정 → AI가 생성 → 자동 검증 → 사람이 채택"의 순환을 모든 작업에 똑같이 적용했다. 핵심은 **검증 게이트**다. AI 산출물은 검증을 통과하기 전까지 저장소에 병합되지 않는다.



저장소에 남는 기록

docs/PROMPTS/ — 사용한 프롬프트 원문과 후처리 절차
docs/AI_USAGE_LOG.md — 날짜별 도구 · 목적 · 사람의 결정 · 검증 결과
docs/ASSET_LICENSES.md · THIRD_PARTY_NOTICES.md — 자산 출처와 라이선스

사용한 AI 도구

도구	담당 영역	사용 방식
OpenAI Codex GPT-5 계열	구현 · 리팩터링 · 테스트	기획서를 규칙 엔진 · 콘텐츠 카탈로그 · 런 상태 · UI 모듈로 분해하고 구현. 48장 구성과 족보 판정의 정합성 검사, 경계값 테스트 생성
Anthropic Claude Cowork	규칙 재설계 · 밸런스 · 문서	포커식 즉시 족보 체계를 전통 놀이 짓고쟁 기반으로 교체. 완전 탐색 프로브를 근거로 목표 곡선 재조정, 튜토리얼 재작성, 제출 문서 정리
OpenAI 이미지 생성	시각 자산	화투 48장 아틀라스를 도트 화풍으로 생성. 부적 · 비결서 · 두목 · 인장 등 카드 자산 제작

세 도구 모두 **개발 단계에서만** 사용했다. 빌드 산출물과 실행 중인 게임에는 어떤 AI 클라이언트도 포함되어 있지 않다.

주요 프롬프트와 지시 사항

프롬프트를 쓸 때 일관되게 지킨 원칙은 "무엇을 만들라"보다 "무엇을 지켜라"를 먼저 쓴다는 것이었다. 제약을 앞에 놓으면 모델이 임의로 규칙을 바꾸거나 구조를 재설계하는 일이 크게 줄었다.

① 구현 지시 — 모듈 경계를 먼저 못 박는다

P R O M P T · docs/PROMPTS/prompt-2026-08-03-build-001.md

기획서에 제안된 시스템을 모두 포함하는 웹게임 프로토타입을 구현한다. 다음 제약을 우선한다.

1. 화투 48장, 점수식, 수집 족보, 고·스톱을 **실제 플레이 가능하게** 구현한다.
2. 파일을 **규칙 엔진 · 콘텐츠 · 런 상태 · UI**로 모듈화한다.
3. 카드 · 부적 · 화공패 · 두목마다 교체 가능한 `assetTag` 를 제공한다.
4. NAN 2026 제출 조건에 맞는 **링크 실행형 웹게임**과 증빙 문서 구조를 유지한다.

2번 제약이 이후 모든 작업의 기준이 됐다. 규칙 엔진(`game/engine/`)이 React를 import하지 않는 순수 TypeScript로 분리되어 있어야 DOM 없이 테스트할 수 있고, 규칙을 바꿔도 UI가 깨지지 않는다.

② 규칙 재설계 지시 — 바꿀 것과 지킬 것을 나눠서 준다

P R O M P T · 짓고땡 전환 지시 (요약)

포커식 즉시 족보 17종을 전통 화투 놀이 **짓고땡** 기반 규칙으로 교체한다.

1. 제출한 패는 항상 **짓**(월 합이 10의 배수)과 **끗패**(셋다 족보 2장)로 갈린다.
2. 원래 5장 고정인 짓고땡을 **2~5장 자유 제출**로 일반화한다. 두 장 제출은 항상 성립시켜 진입 장벽을 낮춘다.
3. **끗패** 판정 우선순위는 **광땡 → 땡 → 이름 붙은 특수패 → 갑오 → 꾀 → 망통**. 이름 붙은 특수패가 숫자 판정을 이긴다.
4. 면제 규칙을 새로 만들지 않는다. 예외는 두 장 제출 하나뿐이다.
5. 규칙 변경으로 의미를 잃은 상태 · 액션 · 콘텐츠는 **남겨 두지 말고 제거**한다.
6. 변경된 규칙을 검증하는 테스트를 함께 작성한다.

5번이 특히 중요했다. 규칙을 바꿀 때 죽은 코드를 남기면 다음 세대의 프롬프트가 그 코드를 근거로 잘못된 추론을 한다. "지우라"를 명시하지 않으면 모델은 기본적으로 보존하는 쪽을 택한다.

③ 밸런스 지시 — 숫자를 감이 아니라 측정으로 정한다

P R O M P T · 목표 곡선 산정 지시 (요약)

목표 점수를 임의로 정하지 말고, **측정값에서 역산**한다.

1. 40개 시드로 부적 없는 런을 재현하는 테스트를 만든다.
2. 각 8장 손패에서 가능한 **모든 2~5장 조합을 완전 탐색**해 최고점을 고르고, 네 손을 냈을 때의 라운드 점수를 잰다.
3. 사람은 완전 탐색이 아니므로 실제 무보정 상한은 그 값의 절반쯤으로 본다.
4. 1~3판은 맨손으로 넘길 수 있게, 4~6판에서 그 상한을 넘게, 7~12판은 배수 엔진만 따라올 수 있는 속도로 잡는다.
5. 목표 곡선을 **테스트가 직접 검증**하게 만들어, 숫자를 손대면 테스트가 깨지도록 한다.

④ 아트 생성 프롬프트 — 원본을 기준으로 못 박는다

화투는 **카드마다 정해진 모티프가 있는 문화 자산**이다. 그림을 자유롭게 그리게 두면 화투가 아닌 것이 나온다. 그래서 원본 아틀라스를 "내용과 기하 구조의 기준"으로 지정하고, 생성 모델에는 **렌더링 화풍만** 바꾸도록 지시했다.

P R O M P T · docs/PROMPTS/prompt-2026-08-06-hwatu-generated-v3.md (원문)

Redraw the complete traditional Korean hwatu atlas as genuine handcrafted pixel art. Treat the original atlas as authoritative for content and geometry: keep exactly 48 cards in the exact 8-column by 6-row order, with the same card boundaries, portrait proportions, traditional motifs, silhouettes, object counts, relative sizes, positions, orientations, negative space, ribbon presence and colors, animals, flowers, branches, moon, rain, and ground shapes. Preserve the red and blue ribbon markings as recognizable brush-calligraphy-like pixel marks. **A veteran Korean hwatu player must identify every card and month immediately.**

Use the earlier January sheet only as a rendering-style reference: strong square pixel clusters, stepped diagonals, crisp hard edges, no antialiasing, restrained Korean minhwa line character, subtle hanji fiber, and a dancheong palette of ink black, warm ivory, vivid red, indigo, pine green, and aged gold. **Do not add scenery or decorations absent from the corresponding original card, and do not reorder, crop, merge, omit, duplicate, or redesign cards.** Output a single flat 8x6 sprite atlas without perspective, labels, mockup, or shadows.

"베테랑 화투 플레이어가 모든 카드와 월을 즉시 알아볼 수 있어야 한다"는 문장이 이 프롬프트의 합격 기준이다. 추상적인 품질 요구("예쁘게", "고퀄리티로") 대신 **판정 가능한 조건**을 쓰면 결과를 채택할지 말지를 사람이 바로 판단할 수 있다.

프롬프트 설계에서 정리된 원칙

원칙	왜	실제 효과
제약을 먼저, 요구를 나중에	모델은 명시되지 않은 부분을 자유롭게 재설계한다	모듈 경계와 판정 우선순위가 세대를 거쳐도 유지됐다
판정 가능한 합격 기준을 쓴다	"좋게 만들어"는 채택 여부를 결정할 수 없다	"화투 플레이어가 즉시 식별", "테스트를 통과" 같은 기준으로 재생성 횟수를 줄였다
지을 것을 명시한다	지시가 없으면 모델은 보존하는 쪽을 택한다	규칙을 교체할 때 죽은 상태·액션·콘텐츠가 남지 않아, 다음 프롬프트가 옛 코드로 오추론하지 않았다
테스트를 산출물에 포함시킨다	코드만 받으면 검증은 사람의 몫으로 남는다	변경마다 검증이 함께 생겨, 규칙 체계를 통째로 바꾸는 작업이 가능해졌다
생성 결과를 결정론적으로 후처리한다	생성 모델의 출력은 픽셀 단위로 불규칙하다	아틀라스 정렬 보정으로 48장 전부가 정확히 2:3 비율을 갖게 됐다

AI 산출물의 후처리와 검증

생성 결과를 그대로 쓰지 않았다. 이미지는 **결정론적 알고리즘으로 보정**했고, 코드는 **자동 테스트를 통과해야만** 채택했다. 덕분에 규칙 체계를 통째로 교체하는 큰 변경도 안전하게 진행할 수 있었다.

이미지 — 결정론적 정렬 보정

이미지 모델이 돌려준 아틀라스는 칸의 너비와 높이가 불규칙했고 쓰이지 않는 검은 여백이 남아 있었다. 이 상태로 CSS 스프라이트를 만들면 48장 전부에서 몇 픽셀씩 잘린다. 그래서 다음 절차로 보정했다.

단계	처리
1	생성된 48개 카드 영역의 실제 경계를 자동 검출 한다
2	각 영역을 최근접 이웃 샘플링 으로 정확히 160×240 픽셀 칸에 맞춘다 (도트 화풍이므로 보간을 쓰지 않는다)
3	여백 없는 정확한 1280×1440 아틀라스로 재조립한다 — 한 칸이 정확히 2:3
4	48칸 전부의 경계를 다시 훑어 여백이 0인지 측정으로 확인 한다
5	게임에는 개별 크롭한 WebP 파생본을 사용하고, 무손실 원본을 저장소에 함께 보존한다

보정 스크립트는 `scripts/slice-hwatu-atlas.py`에 있다. 같은 입력에 같은 출력이 나오는 결정론적 처리다.

코드 — 통과하지 못하면 남지 않는다

```
$ npm run verify # 테스트 → 타입 검사 → 린트 → 프로덕션 빌드
```

자동 테스트 124 개 Vitest · DOM 없이 실행	테스트 코드 2,672 줄 로직 코드의 36%	타입 검사 strict TypeScript 5.9	런타임 AI 호출 0 회 의존성 자체가 없음
--	--	--	---------------------------------------

불변식으로 잠근 것들

AI가 코드를 고칠 때 가장 위험한 것은 두 곳에 나뉘어 있는 같은 규칙이다. 한쪽만 고쳐도 타입 검사와 린트는 통과하기 때문에, 이런 관계는 테스트로 직접 못 박았다.

불변식	잠그는 테스트	깨지면 생기는 일
화면 ≡ 채점기 수집판이 그리는 카드 = 채점기가 세는 카드	<code>collection-board.test.ts</code>	수집판이 채점기가 세지 않는 카드를 약속한다. 줄을 채웠는데 보상이 안 나온다
목표 곡선 ≡ 측정값	<code>balance-probe.test.ts</code>	밸런스가 리뷰 없이 흘러간다. 중반이 갑자기 벽이 되거나 반대로 무의미해진다
콘텐츠 선언 ≡ 실제 <code>CONTENT_EXPECTED ≡ CONTENT_ACTUAL</code>	<code>content.test.ts</code>	콘텐츠를 지우거나 추가하면서 상점 가중치와 문서가 조용히 어긋난다
카드 연출 채널의 배타성 각인은 재질, 판본은 표면	<code>card-visuals.test.ts</code>	한쪽이 다른 쪽을 덮어써서, 산 강화가 화면에 아무것도 표시되지 않는다
점수 연산의 순서 가산 → 곱셈, 롤백 시 원상 복구	<code>engine.test.ts</code> <code>state.test.ts</code>	같은 패가 상황에 따라 다른 점수를 내거나, 고 실패 후 상태가 되돌아가지 않는다

외부 에셋 · 오픈소스 출처

시각 자산

범주	수량	출처 · 제작 방식	권리
화투 48장	48장 + 아틀라스	사용자 제공 원본 아틀라스를 구조 기준으로 삼아 OpenAI 이미지 생성 (화풍 변환)	프로젝트용 생성 자산. 원본 이용 권리는 참가자가 확인 완료
부적	58장	OpenAI 이미지 생성	프로젝트용 생성 자산
비결서 · 화공패 · 금단패 · 뭍음	2:3 세로 카드	OpenAI 이미지 생성	프로젝트용 생성 자산
두목	패패형 아트	OpenAI 이미지 생성	프로젝트용 생성 자산
카드 효과 · 강화 · 판본 · 인장	정사각 오버레이	OpenAI 이미지 생성 + 자체 도트 SVG(CardMark.tsx)	프로젝트용 생성 자산 / 자체 제작
UI 장식 · 이펙트	—	자체 HTML/CSS/SVG 코드	프로젝트 자체 제작

파일 경로와 생성 방식은 `assets/manifest.json`에 정리되어 있다. 구매하거나 내려받은 유료·무료 스톡 이미지는 사용하지 않았다.

폰트

폰트	제작	라이선스	비고
Galmuri 11 Galmuri 11 Bold Galmuri 14	quiple	SIL Open Font License 1.1	github.com/quiple/galmuri 라이선스 사본: <code>public/assets/fonts/GALMURI-0FL.txt</code> 폰트 파일 자체를 판매하지 않는 범위에서 게임에 포함 · 재배포 가능

오픈소스 라이브러리

패키지	버전	라이선스	용도
React · React DOM	19.2.6	MIT	UI 렌더링
Next.js	16.2.6	MIT	App Router 규약
vinext	0.0.50	MIT	Next App Router를 Vite/RSC 위에서 구동
Vite	8.0.13	MIT	번들러 · 개발 서버
Tailwind CSS	4.2.1	MIT	레이아웃 유틸리티
@cloudflare/vite-plugin · Wrangler	1.37.1 · 4.92.0	MIT / Apache-2.0	Cloudflare Workers 빌드 · 배포
TypeScript	5.9.3	Apache-2.0	정적 타입 검사
Vitest	4.0.18	MIT	자동 테스트
ESLint · eslint-config-next	9.39.4 · 16.2.6	MIT	정적 분석

정확한 의존성 트리와 버전은 저장소의 `package-lock.json`이 기준이다. 고지 문서는 `docs/THIRD_PARTY_NOTICES.md`에 유지한다.

권리 및 재현성 고지

항목	내용
런타임 AI	사용하지 않음. 게임 실행 중 어떤 생성형 AI · 유료 API도 호출하지 않는다
실행 요건	API 키 · 계정 · 유료 라이선스가 필요 없다. 심사자가 링크만으로 실행할 수 있다
재현성	모든 런은 표시된 문자열 시드와 RNG 커서로 완전히 재현된다
프롬프트 기록	<code>docs/PROMPTS/</code> — 사용한 프롬프트 원문과 후처리 절차
활용 로그	<code>docs/AI_USAGE_LOG.md</code> — 날짜별 도구 · 목적 · 사람의 결정 · 검증 결과
자산 장부	<code>docs/ASSET_LICENSES.md</code> · <code>assets/manifest.json</code>
커밋 기록	GitHub 저장소에 전체 이력 유지 — github.com/lshane1028/nhh-2026-hackathon